

Módulo 1

Python Profesional para Pronóstico

De "scripts" aislados a sistemas de ingeniería reproducibles y
mantenibles

| Objetivos de Aprendizaje



Reproducibilidad

Garantizar que el código funcione igual en cualquier entorno mediante gestión de dependencias.



Mantenibilidad

Arquitectura modular (Object Oriented Programming) que permite escalar y cambiar modelos sin romper el sistema.



Colaboración

Uso profesional de Git para trabajar en equipos multidisciplinarios con trazabilidad total.

EL PROBLEMA DEL "MEGA-SCRIPT"

EL ALGORITMO DE CÁLCULO ELÉCTRICO

ESTADO INICIAL

Imagina un script de 2,000 líneas para calcular la carga de una planta industrial. Todo vive en un solo archivo: constantes, fórmulas y datos.

- **Rigidez:** Cambiar un cable implica revisar 50 funciones.
- **Opacidad:** Difícil de leer por otros ingenieros.

CONSECUENCIAS

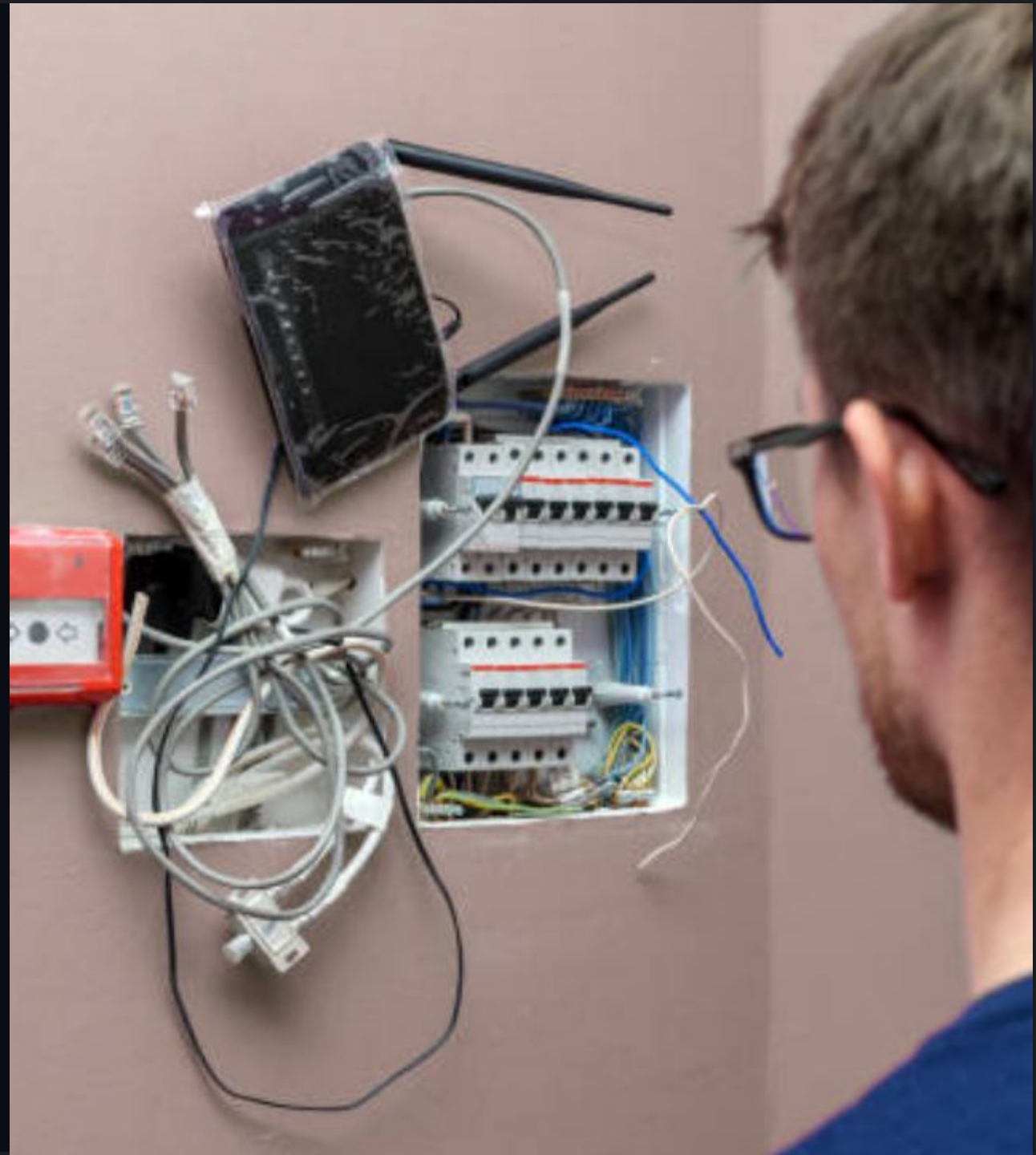
Cualquier modificación mínima genera errores en cascada. Es como tener un tablero eléctrico sin etiquetas ni modularidad.

```
ERROR: Variable 'voltaje_3' no definida en línea 1542.
```

EL COSTO DE LA RIGIDEZ

Programar sin objetos es como el cableado de la izquierda: funcional pero **imposible de mantener**.

La Programación Orientada a Objetos busca la estructura de la derecha: **módulos independientes** y bien definidos.



PENSAR EN COMPONENTES

CLASE VS OBJETO



LA CLASE ES EL DISEÑO

En ingeniería, no fabricas sin un plano. La **Clase** es el plano técnico que define qué atributos (especificaciones) y qué métodos (funciones) tendrá un componente.

Ejemplo: La clase Motor define que todos tienen potencia y velocidad.

| EL OBJETO: LA PIEZA REAL

DEL PLANO A LA FÁBRICA

Un **Objeto** es la instancia física. Es el motor real que instalas en la cinta transportadora.

- Puedes crear 100 motores (objetos) a partir del mismo plano (clase).
- Cada uno tiene su propio estado (un motor está ON, otro a 50Hz).



OTROS BENEFICIOS DE ESTA FORMA DE PROGRAMAR



ENCAPSULAMIENTO

Protección de datos internos.
Nadie toca los engranajes
directamente.



HERENCIA

Reutilización de diseños base
para crear versiones
especializadas.



POLIMORFISMO

Misma interfase para diferentes
comportamientos técnicos.

ENCAPSULAR: "CAJA NEGRA"

Un ingeniero no necesita saber cómo funciona el silicio de un sensor para usarlo. Solo necesita saber que si le da voltaje, recibe una señal.

En POO, ocultamos la complejidad interna y solo exponemos **Interfaces de Control**.



HERENCIA: FAMILIAS DE DISEÑO

JERARQUÍAS TÉCNICAS

Todos son "Aparatos Eléctricos" (Clase Base), pero un Refrigerador es diferente a un Horno.

- **Clase Base:** Voltaje, Consumo.
- **Especialización:** El Refrigerador hereda el voltaje, pero añade "Temperatura".



POLIMORFISMO: EL ESTÁNDAR

INTERFASE UNIVERSAL

Imagina un enchufe universal. No importa si conectas una lámpara o un taladro, ambos "saben" cómo consumir energía.

El código le dice al objeto `calcular_consumo()`, y cada objeto lo hace a su manera sin que el sistema principal sepa los detalles.



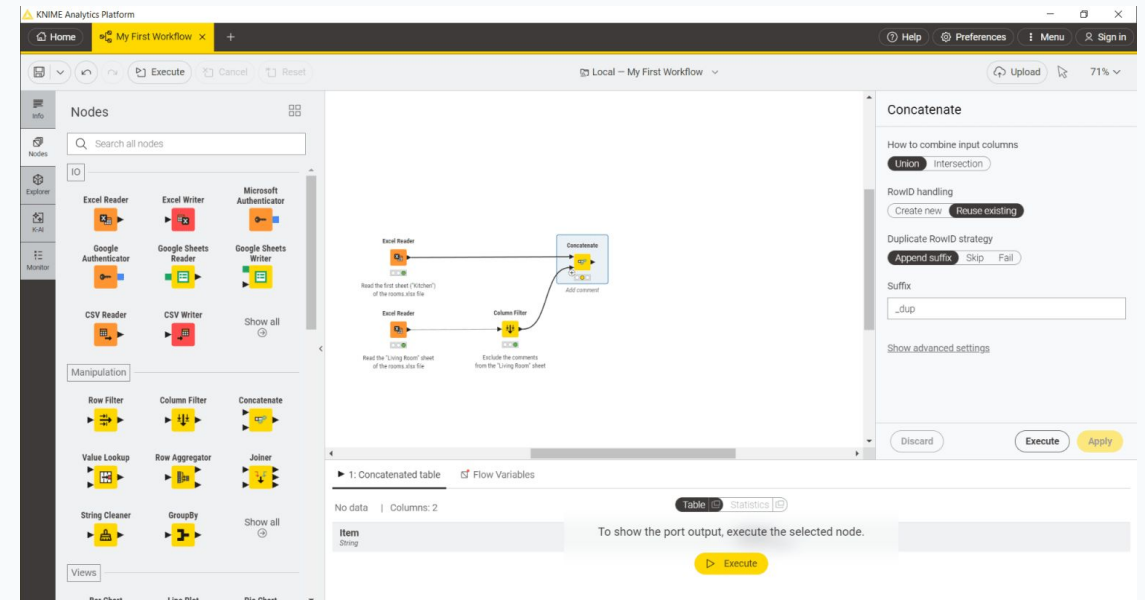
Analogía con Knime

El problema no es que el script falle

El problema es que nadie puede revisar con confianza qué hizo, con qué datos y bajo qué parámetros.

En KNIME

Un flujo grande sin nombres claros, nodos duplicados y configuraciones escondidas se vuelve difícil de auditar.



Meta de la sesión: transformar código que solo corre en un sistema que otro ingeniero puede entender, ejecutar y revisar.

Meta de la sesión: transformar código que solo corre en un sistema que otro ingeniero puede entender, ejecutar y revisar.

KNIME y Python pueden describir el mismo flujo

Entrada

Tabla con temperatura, día y datos disponibles antes del pronóstico.

Nodo

Regla configurada para una subestación específica.

Salida

Demanda estimada y rastro de cómo se calculó.

En KNIME lo vemos como cajas conectadas. En Python lo escribimos como datos que entran, métodos que procesan y resultados que salen.

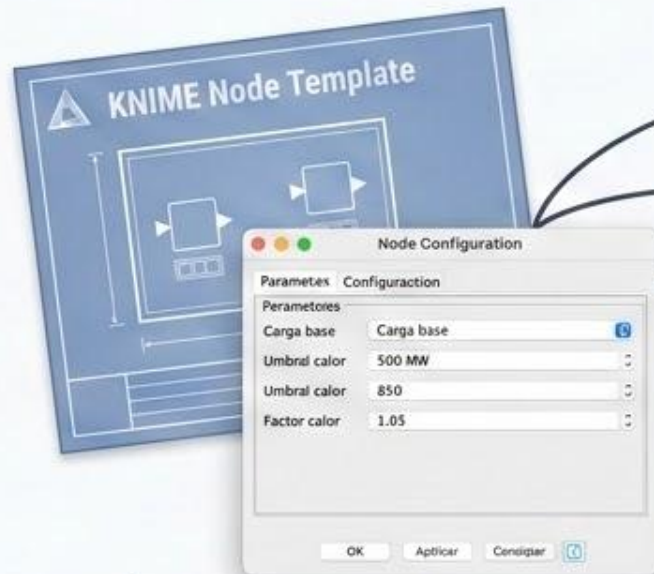
Clase significa definición de nodo

Clase

Plantilla de Nodo

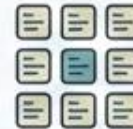


Definición del nodo: qué configuraciones pide y qué operación sabe ejecutar.

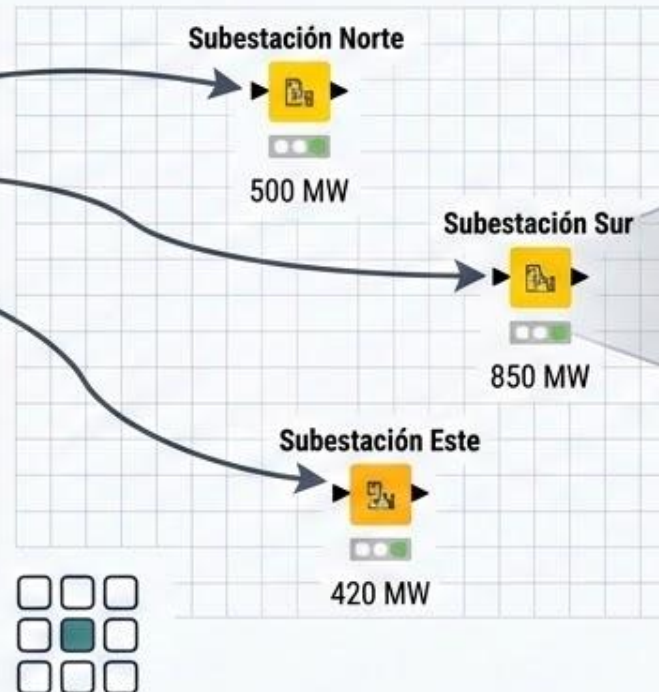


Objeto

Nodo Específico en Lienzo



Un nodo específico ya arrastrado al lienzo: Subestación Norte, Sur o Este.



Encapsulamiento

Protección de Fórmula



Lo que vive dentro del nodo y su ventana de configuración. El usuario no toca la fórmula dispersa.



Un nodo de pronóstico tiene cuatro partes

Configuración

Carga base, umbral de calor y factor por temperatura.

Entrada

Una fila con datos conocidos antes del pronóstico.

Proceso

La regla interna que calcula la demanda.

Salida

Demanda estimada más una explicación mínima.

El script desastroso mezcla todo en el mismo lienzo

```
for temp, dia in datos_clima:
    carga_base_norte = 500
    if dia == "Domingo" or dia == "Sabado":
        demanda = carga_base_norte * 0.75
    if temp > 30:
        demanda = demanda + (temp - 30) * 12.5
    print(demanda)
```

- La configuración está escondida dentro del ciclo.
- La regla se copia para cada subestación.
- No queda claro qué dato era conocido antes del pronóstico.
- La revisión depende de leer todo el archivo.

Los 4 pilares vuelven auditable el flujo

Reproducibilidad

Mismos datos y mismos parámetros producen el mismo resultado.

Evitar Data Leakage

El nodo solo usa columnas disponibles antes del momento a pronosticar.

Sanity Checks

Antes de confiar, revisamos que entradas y salidas tengan sentido físico.

Trazabilidad

Cada resultado conserva qué nodo, parámetros y fila lo generaron.

Reproducibilidad es configurar el nodo de forma visible

```
norte = NodoDemandaSubestacion(  
    nombre="Norte",  
    carga_base_mw=500,  
    umbral_calor_c=30,  
    factor_calor_mw=12.5,  
    factor_fin_semana=0.75  
)
```

Como en KNIME, la configuración debe poder verse, repetirse y explicarse. Si el parámetro está escondido en una fórmula, el flujo deja de ser confiable.

El objeto es el nodo concreto en el lienzo

Una sola definición de nodo permite crear varias subestaciones con configuraciones distintas.

Norte

Carga base 500 MW
Umbral 30 C
Factor 12.5

Sur

Carga base 850 MW
Umbral 32 C
Factor 15.1

Este

Carga base 420 MW
Umbral 28 C
Factor 10.0

En KNIME serían tres nodos del mismo tipo, cada uno con su propia ventana de configuración.

Encapsular es dejar la fórmula dentro del nodo

```
def procesar_fila(self, fila):  
    demanda = self.carga_base_mw  
  
    if fila["dia"] in ["Sabado", "Domingo"]:  
        demanda = demanda * self.factor_fin_semana  
  
    if fila["temperatura_c"] > self.umbral_calor_c:  
        exceso = fila["temperatura_c"] - self.umbral_calor_c  
        demanda = demanda + exceso * self.factor_calor_mw  
  
    return demanda
```

El flujo principal ya no toca la fórmula. Solo entrega una fila al nodo y recibe una salida.

Evitar leakage es cerrar la puerta al futuro

Permitido

Temperatura pronosticada, día de la semana, calendario conocido, configuración de la subestación.

Prohibido

Demanda real del mismo día, errores calculados después, variables que solo existen al cierre de operación.

Traducción KNIME: antes del nodo de pronóstico, el flujo debe contener solo columnas disponibles en ese momento.

Sanity checks son pruebas de sentido físico

- La temperatura debe estar en un rango posible.
- El día debe pertenecer al calendario esperado.
- La carga base no puede ser negativa.
- La demanda estimada debe tener unidades y magnitud razonables.

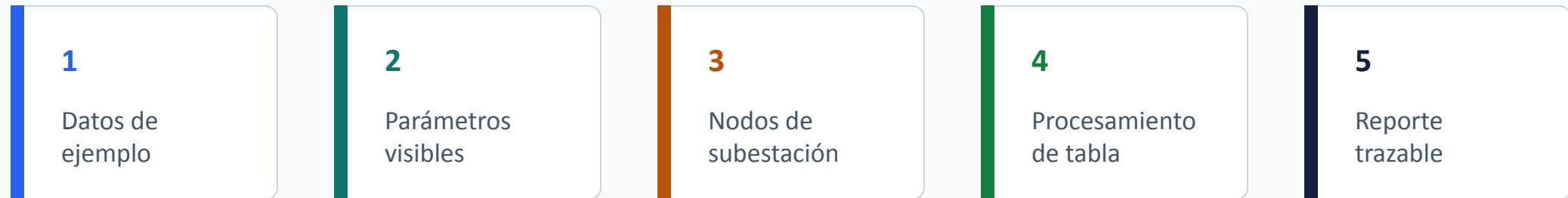
No reemplazan al modelo. Evitan que un resultado absurdo viaje por todo el flujo.

Trazabilidad es dejar rastro de cada nodo

```
{  
  "subestacion": "Norte",  
  "dia": "Martes",  
  "temperatura_c": 34,  
  "demanda_mw": 550.0,  
  "regla": "base + ajuste_calor"  
}
```

Cuando alguien pregunte de dónde salió el número, el resultado debe contestar: qué nodo lo generó, con qué entrada y bajo qué configuración.

El flujo final se parece a KNIME, pero vive en Python



La diferencia es que ahora el flujo se puede versionar, revisar línea por línea y ejecutar igual en otra computadora.

Ejercicio: agreguen una subestación sin tocar la fórmula

Reto de 10 minutos

- Crear un nuevo objeto para la subestación Oeste.
- Definir su carga base, umbral y factor de calor.
- Procesar la misma tabla de clima.
- Verificar que el reporte conserve trazabilidad.

Pregunta de cierre

¿Qué tuvieron que cambiar: la configuración del nodo o la lógica interna?

¿Dudas?